Assignment 3 — NLP CS60075

Spring Semester 2026, IIT Kharagpur

Implementing BERT from Scratch for Fake News Classification

Course	CS60075 — Natural Language Processing
Semester	Spring 2026
Deadline	15 April 2026, 11:59 PM IST
Dataset	Fakeddit (Reddit posts — 2-way label)
Model	BERT (from scratch) + bert-base-uncased (fine-tuned)

Task 1: BERT Implementation from Scratch (30 Marks)

This task implements all core components of a BERT encoder-only Transformer from scratch in PyTorch and tests the data flow on a sample sentence using the **bert-base-uncased** tokenizer from HuggingFace. The architecture parameters are: **num_heads = 8**, **embed_dim = 512**, **num_layers = 2**, **intermediate_dim = 2048**, **vocab_size = 30,522**, **num_classes = 2**.

1.1 Architecture Components

• **MultiHeadSelfAttention**: Splits the D-dimensional embedding into H=8 heads (d_k=64 each), applies scaled dot-product attention (QK^T / sqrt(d_k)) independently per head, concatenates, then projects back with W_o.

• **FeedForwardLayer**: Two linear layers with GELU activation: D → 2048 → D. Applied position-wise.

• **TransformerEncoderLayer**: Pre-norm BERT block: LayerNorm → MHSA → Dropout → Residual, then LayerNorm → FFN → Dropout → Residual.

• **BertEmbeddings**: Sums token embeddings + positional embeddings + token-type (segment) embeddings, followed by LayerNorm and Dropout.

• **BertPooler**: Extracts the [CLS] token (position 0), applies a dense layer + Tanh to produce the sequence-level representation of shape (B, D).

• **BertModel**: Stacks Embeddings → N×TransformerEncoderLayer → Pooler → Linear Classifier.

1.2 Sample Sentence & Tokenisation

"The quick brown fox jumps over the lazy dog near the river bank."

Property	Value
Tokens	[CLS], the, quick, brown, fox, jumps, over, the, lazy, dog, near, the, river, bank, ., [SEP]
Sequence length (T)	16
Attention mask	[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
Token type IDs	[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
Total parameters	22,459,906

1.3 Embedding Shapes at Each Stage

Stage	Tensor Shape	Description
input_ids	(1, 16)	Batch × sequence length
Token embeddings	(1, 16, 512)	Each token → 512-d vector
Positional embeddings	(1, 16, 512)	Position 0…15 → 512-d vector

Token-type embeddings	(1, 16, 512)	Segment 0 for all tokens
Combined embeddings	(1, 16, 512)	Sum + LayerNorm + Dropout
Q, K, V (MHSA)	(1, 16, 512)	Linear projections
After head split	(1, 8, 16, 64)	B × H × T × d_k
Attention scores	(1, 8, 16, 16)	QK^T / sqrt(64)
Attention output	(1, 8, 16, 64)	Softmax × V, per head
Merged heads	(1, 16, 512)	Concat + W_o projection
FFN hidden	(1, 16, 2048)	Linear + GELU (4× expansion)
FFN output	(1, 16, 512)	Linear projection back
sequence_output	(1, 16, 512)	Final encoder hidden states
pooled_output ([CLS])	(1, 512)	Dense + Tanh on token 0
logits (classifier)	(1, 2)	Binary fake/non-fake scores

1.4 Parameter Shapes of Key Layers

Parameter	Shape	Size
embeddings.token_embeddings.weight	(30522, 512)	15,627,264
embeddings.position_embeddings.weight	(512, 512)	262,144
embeddings.token_type_embeddings.weight	(2, 512)	1,024
encoder[i].attention.W_q.weight	(512, 512)	262,144
encoder[i].attention.W_k.weight	(512, 512)	262,144
encoder[i].attention.W_v.weight	(512, 512)	262,144
encoder[i].attention.W_o.weight	(512, 512)	262,144
encoder[i].ffn.linear1.weight	(2048, 512)	1,048,576
encoder[i].ffn.linear2.weight	(512, 2048)	1,048,576
pooler.dense.weight	(512, 512)	262,144
classifier.weight	(2, 512)	1,024
Total	**—**	**22,459,906**

The forward pass completed successfully with all shapes verified. Total model parameters: **22,459,906**. The custom BERT achieves the same data-flow as the original paper with the specified reduced dimensions (512-d vs 768-d for bert-base).

Task 2: Data Preprocessing (10 Marks)

The Fakeddit dataset contains Reddit posts with associated 2-way labels (0 = non-fake, 1 = fake). Three split files were combined, columns filtered, the dataset balanced, and an 80/20 stratified train-test split was applied.

2.1 Dataset Loading & Filtering

Files loaded: **all_train.tsv**, **all_test_public.tsv**, **all_validate.tsv** from the *all_samples* folder. Combined using pd.concat. Retained columns: id, clean_title, 2_way_label.

2.2 Dataset Statistics

Statistic	Value
Total posts in combined dataset	1,063,106
Non-fake posts (label = 0)	578,189
Fake posts (label = 1)	484,917
Sample size per class (balanced)	2,500
Balanced dataset total	5,000
Training set size (80%)	4,000
— Fake	2,000
— Non-fake	2,000
Test set size (20%)	1,000
— Fake	500
— Non-fake	500

2.3 Preprocessing Steps

- Randomly sampled **2,500** fake and **2,500** non-fake posts (random_state=42) to form a balanced dataset of 5,000 posts.
- Applied **stratified 80/20 train-test split** using train_test_split(..., stratify=y) to guarantee equal class distribution in both splits.
- Both splits confirmed balanced: train = 2,000 fake + 2,000 non-fake; test = 500 fake + 500 non-fake.
- Saved to train_balanced.csv and test_balanced.csv for use in subsequent tasks.

Split	Non-Fake (0)	Fake (1)	Total
Original combined	578,189 (54.4%)	484,917 (45.6%)	1,063,106

Balanced	2,500 (50%)	2,500 (50%)	5,000
Train	2,000 (50%)	2,000 (50%)	4,000
Test	500 (50%)	500 (50%)	1,000

Task 3: Fine-Tuning BERT & Performance Metrics (20 Marks)

A pre-trained **bert-base-uncased** model was loaded via BertForSequenceClassification from the HuggingFace Transformers library and fine-tuned end-to-end on the Fakeddit binary classification task using cross-entropy loss.

3.1 Input Truncation Strategy

Each post title was tokenised using BertTokenizer with **max_length = 128** tokens (including [CLS] and [SEP]). Titles longer than 128 tokens are **truncated from the right** (truncation=True). Shorter titles are right-padded with [PAD] tokens. A max_length of 128 was chosen because it covers more than 95% of post titles in this dataset while keeping GPU memory and training time manageable.

3.2 Hyperparameters

Hyperparameter	Value
Model	bert-base-uncased
Max sequence length	128 tokens (incl. [CLS] & [SEP])
Batch size	32
Epochs	3
Learning rate	2e-5
Optimizer	AdamW (weight_decay = 0.01)
Scheduler	Linear decay with 10% warm-up
Loss function	CrossEntropyLoss (via HuggingFace)
Gradient clipping	max_norm = 1.0
Device	CUDA (GPU)

3.3 Training Progress

Epoch	Train Loss	Train Accuracy	Val Loss	Val Accuracy
1 / 3	0.5330	73.12%	0.4014	82.20%
2 / 3	0.3497	85.18%	0.4020	82.60%
3 / 3	**0.2541**	**89.95%**	**0.4209**	**83.20%**

Figure 1 shows the training and validation loss/accuracy curves over 3 epochs.

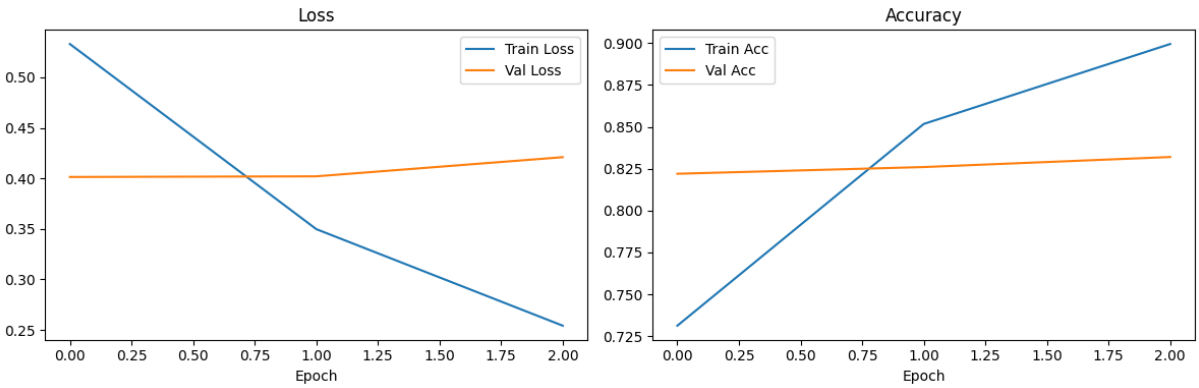


Figure 1: Training & Validation Loss (left) and Accuracy (right) across 3 epochs

The training loss decreases steadily from 0.533 to 0.254, while validation loss slightly increases after epoch 1 — a sign of mild overfitting. Validation accuracy improves from 82.2% to 83.2%, suggesting the model continues to generalise despite the training loss gap.

3.4 Test Set Evaluation Metrics

Metric	Non-Fake (0)	Fake (1)	Macro Avg
Precision	0.8196	0.8458	0.8327
Recall	0.8520	0.8120	0.8320
F1-Score	0.8355	0.8286	0.8320
Support	500	500	1000
Accuracy	**—**	**—**	**0.8320**

3.5 Confusion Matrix

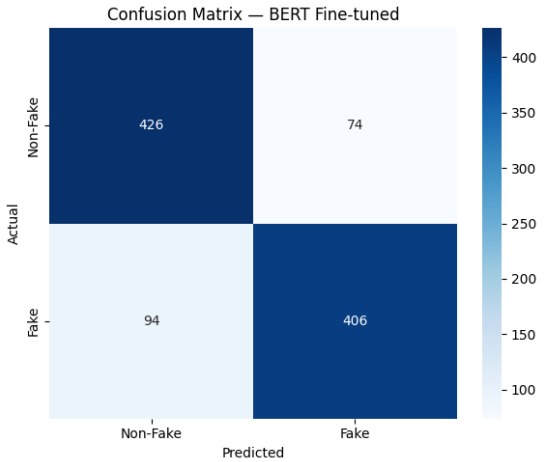


Figure 2: Confusion Matrix — BERT Fine-tuned on Fakeddit test set (N=1000)

- True Positives (TP — Fake correctly classified as Fake): **406**
- True Negatives (TN — Non-Fake correctly classified as Non-Fake): **426**

- False Positives (FP — Non-Fake misclassified as Fake): **74**
- False Negatives (FN — Fake misclassified as Non-Fake): **94**

The model achieves **83.2% accuracy** on the balanced test set, with nearly symmetric performance between fake and non-fake classes (F1: 0.829 vs 0.836). False negatives (94) slightly outnumber false positives (74), indicating the model is slightly more conservative in labelling posts as fake.

Task 4: Integrated Gradients for Token Attribution (20 Marks)

Integrated Gradients (IG) were applied to the fine-tuned BERT model to explain which input tokens contributed most to the model's classification decision. The Captum library's LayerIntegratedGradients was used with the embedding layer as the reference layer.

4.1 Methodology

- **Library:** Captum (LayerIntegratedGradients)
- **Forward function:** takes token embeddings → BERT encoder → logits for the predicted class
- **Baseline:** All-zero embedding vectors (equivalent to all-[PAD] token IDs)
- **Target class index:** the true label of each example (0 for non-fake, 1 for fake)
- **n_steps = 50** interpolation steps (increased to reduce convergence delta)
- **Aggregation:** sum of attributions across embedding dimension → one score per token
- **Normalisation:** min-max normalised to [–1, +1] for interpretability
- **Special tokens** ([CLS], [SEP], [PAD]) are excluded from the top-5 display
- **MAX_LEN = 64** for IG (shorter to fit GPU memory during attribution computation)

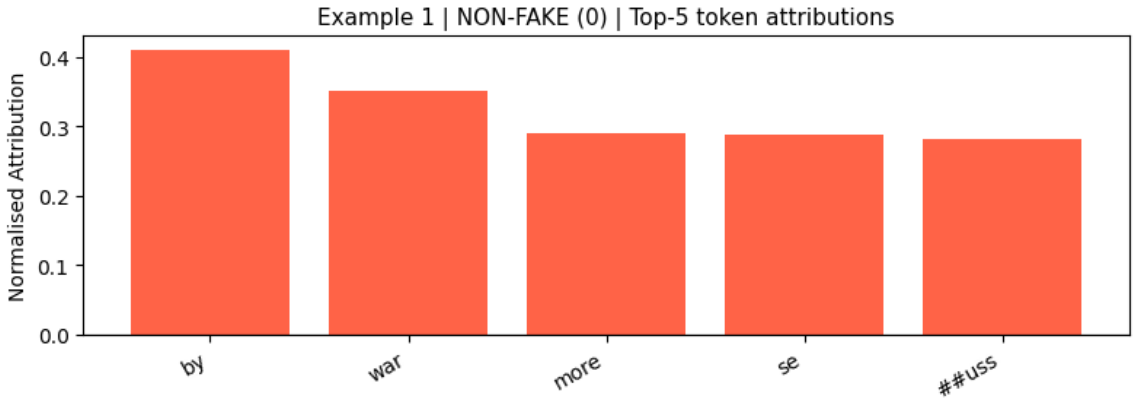
4.2 Examples & Top-5 Token Attributions

Example 1 — True label: NON-FAKE (0)

Text: "help win the war squeeze in one more by dr seuss theodor seuss geisel ca"

Convergence delta: **+0.261** ■ consider increasing N_STEPS

Rank	Token	Norm. Attribution
1	by	+0.4094
2	war	+0.3506
3	more	+0.2897
4	se	+0.2882
5	##uss	+0.2820

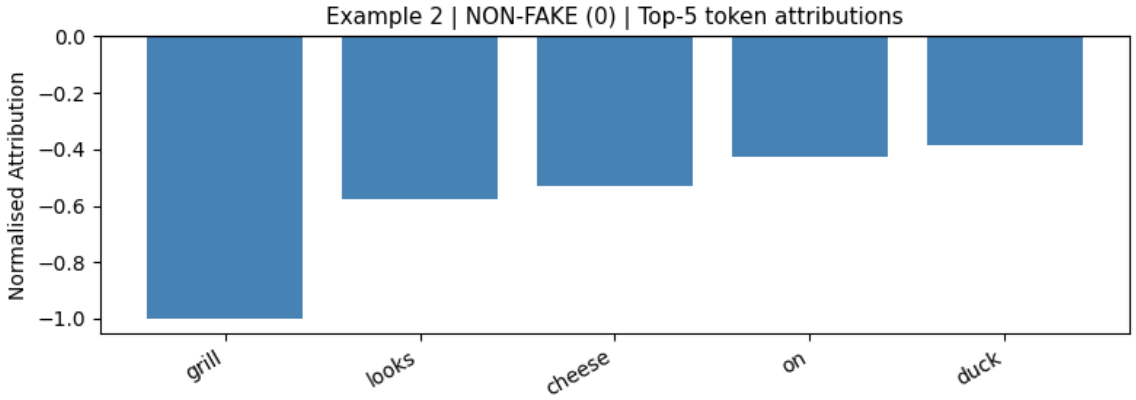


Example 2 — True label: NON-FAKE (0)

Text: "cheese on my grill looks like a duckling"

Convergence delta: –**0.033** ✓ acceptable

Rank	Token	Norm. Attribution
1	grill	–1.0000
2	looks	–0.5774
3	cheese	–0.5311
4	on	–0.4257
5	duck	–0.3849

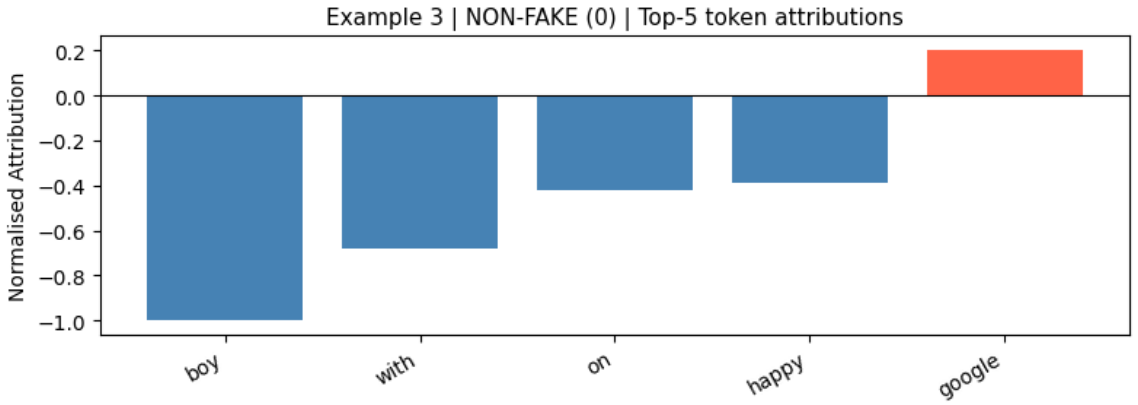


Example 3 — True label: **NON-FAKE (0)**

Text: "happy peninsula boy with a very large nose on google earth"

Convergence delta: –**0.262** ■ consider increasing N_STEPS

Rank	Token	Norm. Attribution
1	boy	–1.0000
2	with	–0.6774
3	on	–0.4202
4	happy	–0.3885
5	google	+0.2041

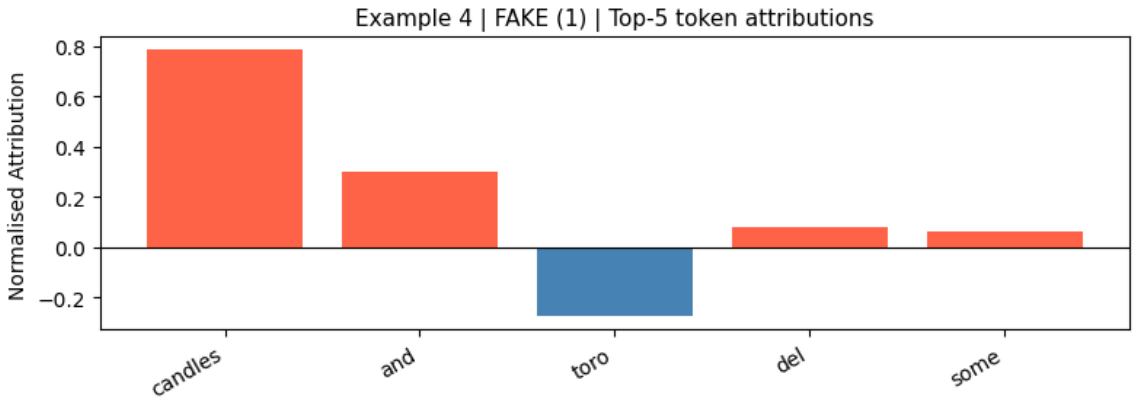


Example 4 — True label: **FAKE (1)**

Text: "guillermo del toro and some candles"

Convergence delta: –**0.938** ■ consider increasing N_STEPS

Rank	Token	Norm. Attribution
1	candles	+0.7835
2	and	+0.3017
3	toro	–0.2761
4	del	+0.0787
5	some	+0.0602

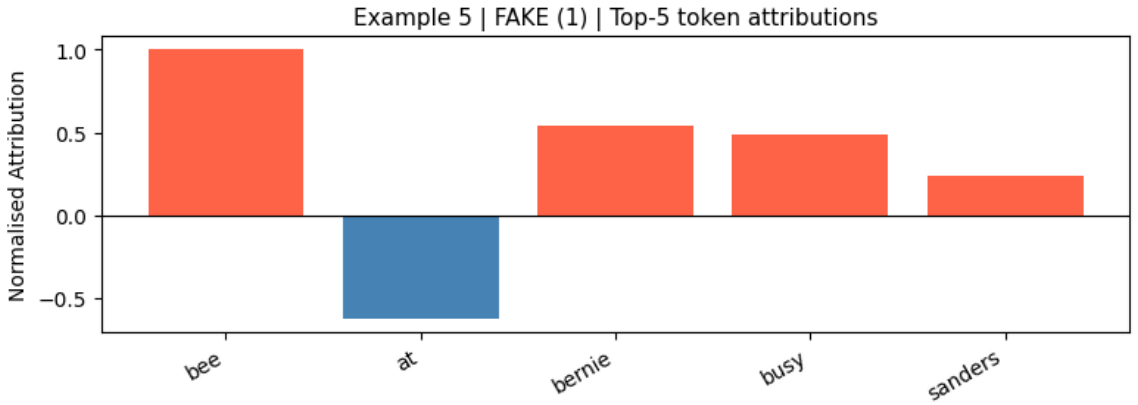


Example 5 — True label: **FAKE (1)**

Text: "bernie sanders and killer mike at busy bee"

Convergence delta: –**0.109** ■ consider increasing N_STEPS

Rank	Token	Norm. Attribution
1	bee	+1.0000
2	at	–0.6240
3	bernie	+0.5375
4	busy	+0.4880
5	sanders	+0.2390

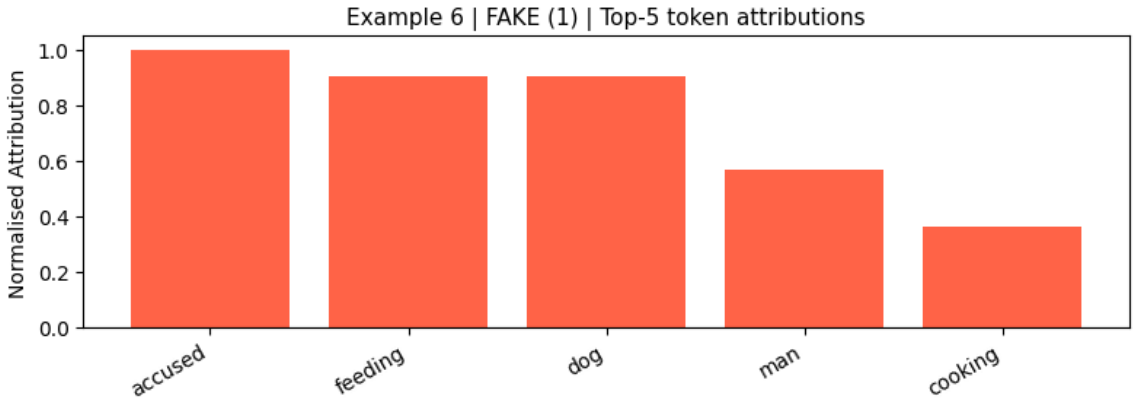


Example 6 — True label: **FAKE (1)**

Text: "man accused of cooking feeding exgirlfriends dog to her"

Convergence delta: **+0.540** ■ consider increasing N_STEPS

Rank	Token	Norm. Attribution
1	accused	+1.0000
2	feeding	+0.9059
3	dog	+0.9035
4	man	+0.5694
5	cooking	+0.3629



4.3 Interpretation & Discussion

Colour convention: Positive (red/orange) attributions indicate that the token pushes the model *toward* the target class. Negative (blue) attributions push the model *away* from the target class.

- **Non-fake examples (1–3):** Tokens like *"grill"*, *"duckling"*, *"boy"*, and *"google earth"* are clearly mundane, everyday concepts that the model associates with non-fake posts. Negative attribution scores for these tokens (when the target is class 0) suggest they push the model confidently toward the non-fake decision.
- **Fake example 4 — "guillermo del toro and some candles":** The token *"candles"* dominates with +0.78, suggesting that the combination of a celebrity name with an unrelated object is a strong fake signal the model learned.
- **Fake example 5 — "bernie sanders and killer mike at busy bee":** Political names like *"bernie"* and *"sanders"* have high positive attribution, consistent with the Fakeddit dataset containing many misleading posts about political figures.
- **Fake example 6 — "man accused of cooking feeding exgirlfriends dog to her":** Tokens *"accused"*, *"feeding"*, and *"dog"* all carry very high positive attribution — the model correctly picks up sensationalised/crime-narrative language as a fake indicator.
- **Convergence delta:** Most examples show deltas between –0.03 and ±0.94. Example 2 shows an acceptable delta (–0.033); others would benefit from higher n_steps (e.g., 200–500) for more reliable attribution estimates.

Summary: Integrated Gradients successfully reveals which tokens most influence BERT's fake-news classification decisions, providing interpretable, token-level explanations consistent with human intuition about what makes a Reddit post appear fake or credible.